

Autonomous Job Application Agent

1. Overview

The Autonomous Job Application Agent turns a **job description** and a candidate's **resume** into three application-ready documents: a tailored resume, a cover letter, and an interview-prep guide. A **PII guardrail** sanitises the resume before any text reaches an LLM; the pipeline then researches the company, extracts the role's requirements, and generates the three documents **in parallel**.

An LLM **critic** scores the outputs and either auto-retries generation (up to 3×) or hands off to a **human-in-the-loop** review gate, where the candidate approves or gives targeted, per-document feedback. Approved documents export to PDF.

Framework	LLM	Core pattern
LangGraph 0.2+ / LangChain	Configurable chat model (env LLM_MODEL)	Multi-agent DAG + parallel fan-out + HITL + PII guard

2. Architecture

The system is a single LangGraph **StateGraph** — a directed graph whose nodes are specialised agents. Every node reads from, and writes partial updates to, one typed **AgentState** object; LangGraph merges each returned dict back into shared state. A **MemorySaver** checkpointer persists state across the human-review interrupt so the run can pause and resume. Execution falls into five tiers:

Tier	Nodes	Pattern
PII Guard	pii_guardrail	First gate — blocks SSN / credit-card / passport; redacts email, phone, address & URLs to reversible tokens before any LLM call
Entry Pipeline	parse_input → research_company → analyze_jd	Sequential — builds the context chain; the web_search tool gathers company intel
Generation	tailor_resume, write_cover_letter, prepare_interview	Parallel fan-out — three independent writers; skip-logic regenerates only the document that got feedback
Quality Gate	aggregate_outputs	Fan-in — LLM critic scores all three docs 0–1 and emits gap feedback; routes to retry or human review
HITL Review	human_review → classify_feedback	Interrupt — human approves or gives per-document feedback; classify_feedback routes to approve / regenerate / re-interrupt

3. Multi-Agent DAG — Execution Flow

The diagram below shows the full graph topology. The PII guardrail intercepts the resume before any LLM call; the orchestrator then delegates to specialised sub-agents — each bound to its own tools — and results fan in to the quality and human-review nodes. **Dashed arrows** are the conditional loops: the quality auto-retry and the per-document feedback re-route.

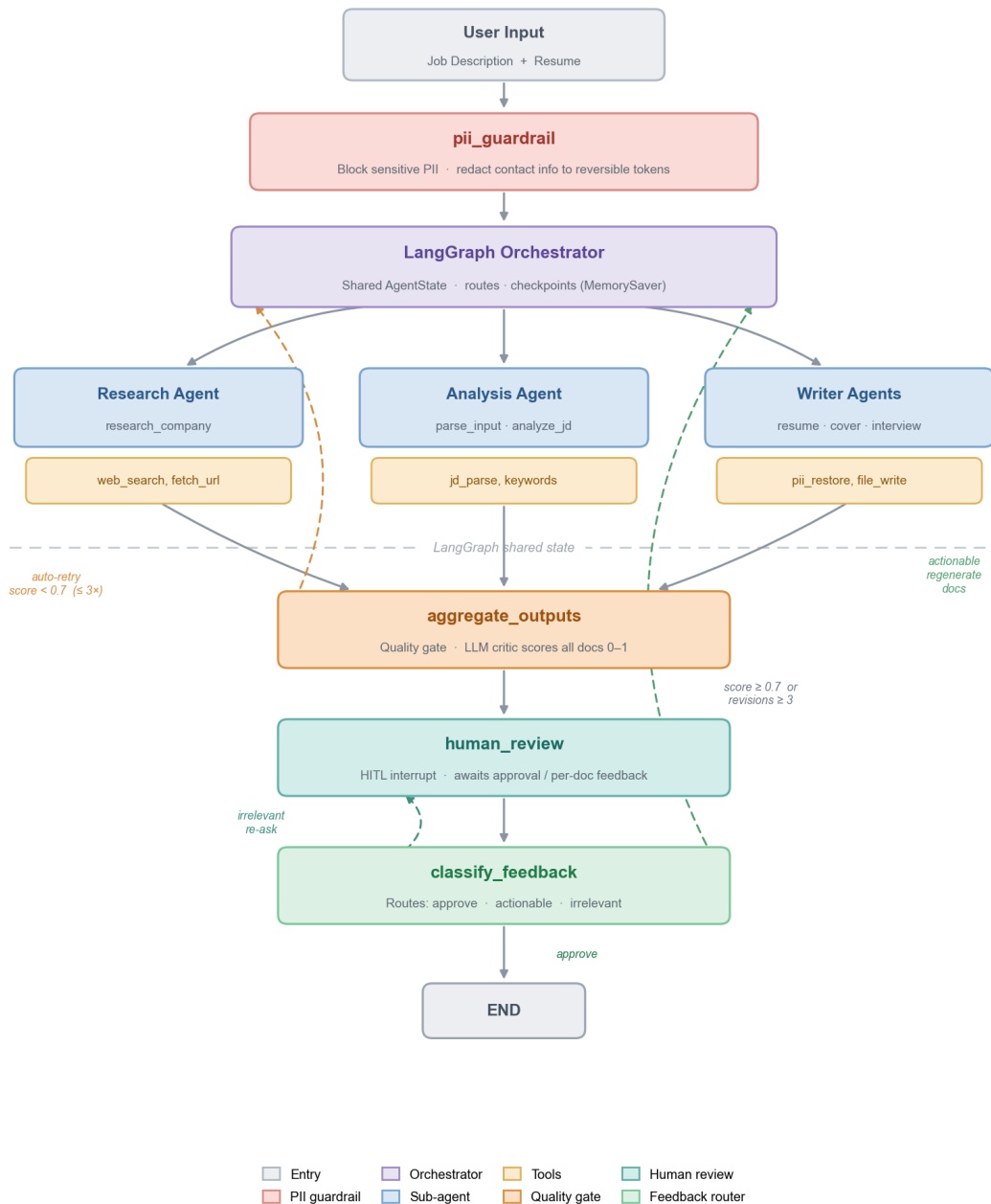


Figure 1 — LangGraph multi-agent DAG: PII guard, research / analysis / writer sub-agents, quality critic, and the human-review feedback loops.

4. Data Flow

4.1 PII Guard Stage

Before any LLM call, the resume passes through `guardrails/pii_guard.py`. Sensitive PII raises a `PIIBlockedError` and halts the pipeline. Standard PII is redacted to opaque tokens; the mapping (*token* → *original value*) is stored in state but never logged.

PII Type	Tier	Action	Token Example
SSN, Credit Card, Passport	Sensitive	BLOCK — pipeline halts	—
Email	Standard	Redact	[PII:EMAIL:a1b2c3]
Phone	Standard	Redact	[PII:PHONE:d4e5f6]
LinkedIn / GitHub URL	Standard	Redact	[PII:LINKEDIN:g7h8i9]
Home Address	Standard	Redact	[PII:ADDRESS:j0k1l2]

4.2 Processing Stage

Node	Temp	Key Input	Key Output
parse_input	0.0	JD + resume	company_name
research_company	0.2	company_name	company_research (web search)
analyze_jd	0.0	JD	jd_analysis (skills, ATS keywords, tone, seniority)
tailor_resume	0.15	sanitized_resume + jd_analysis	tailored_resume (PII restored)
write_cover_letter	0.6	sanitized_resume + company_research	cover_letter (PII restored)
prepare_interview	0.4	sanitized_resume + jd_analysis	interview_questions
aggregate_outputs	0.0	all 3 docs + JD	quality_score, quality_feedback

4.3 PII Restoration

After each generation node, `restore_pii(output, pii_mapping)` swaps tokens back to real values, and `strip_stray_pii_tokens(output)` removes any `[PII:*)` tokens the LLM hallucinated. The LLM never sees real contact details; final documents have them fully restored. On revision, each writer is fed its own *tokenised* previous draft, so it edits its prior output without ever touching raw PII.

4.4 HITL & Feedback Routing

At the **human_review** interrupt the UI surfaces the three documents. `classify_feedback` routes on deterministic signals first, LLM classification last. `revision_count` increments only on actionable feedback, so irrelevant input never inflates the retry budget.

Signal (priority order)	Detection	Route
Explicit approval flag	approved=True from the UI's Approve action	→ END (the only way UI feedback finishes the graph)
Per-document feedback, checked individually	Each non-empty box relevance-checked against its own document	→ regenerate only the actionable docs; off-topic text is cleared & re-asked
Free-form feedback (CLI)	Keyword + LLM classification	→ approve · actionable · re-interrupt

5. Quality Assurance & Observability

5.1 In-Agent Quality Loop

The `aggregate_outputs` node runs an LLM critic (temp = 0.0). Scores below 0.7 trigger an automatic retry (max 3×); after that the graph always proceeds to human review regardless of score.

Condition	Route
quality_score ≥ 0.7	→ human_review
quality_score < 0.7 AND revisions < 3	→ auto-retry (back to generation tier)
revisions = 3 (regardless of score)	→ human_review (retry budget exhausted)

5.2 External Evaluation Suite (evals/)

Evals run through the **real API** (submit → poll → grab outputs at the interrupt), so they exercise the exact handlers and background graph that production serves.

Eval	Type	Gate	What it checks
No Fabrication	GEval	0.8	Every claim in the tailored resume traces to the original — the system's core guarantee
Faithfulness	deepeval	0.8	Tailored-resume claims grounded in the original resume
JD Alignment	GEval	0.8	Resume surfaces JD skills / ATS keywords the candidate genuinely has
Cover Letter Quality	GEval	0.8	Specific hook, real experience, no banned clichés, concise
No stray PII tokens	deterministic	pass	Final docs contain no leftover [PII:.*] tokens
Error-handling suite	deterministic	pass	Adversarial inputs fail correctly & safely: SSN halts at the guard (no echo); empty resume/JD → HTTP 400. LLM-free, so free on every push; known gaps tracked as xfail

5.3 Observability

A `UsageTracker` callback attributes **token usage to each LangGraph node**, and every status event is stamped with a monotonic clock so evals derive **per-node latency** from the gaps — exposing where a run spends time and tokens.

6. Key Design Trade-offs

Decision	Why / what we gained	Cost accepted
Regex PII guard with reversible tokens + hard block on sensitive PII	Deterministic, fast, free; runs before any network call so the LLM never sees raw contact details	Limited recall (undashed SSN, JD-side SSN) — accepted and tracked as xfail rather than over-engineered
Parallel fan-out of 3 writers with per-node skip logic	Cuts wall-clock latency ~3×; revisions touch only the document that got feedback	Extra skip / branch logic and shared-state merge discipline in each node
Per-node temperature (0.0 → 0.6); resume held at 0.15	Factual extraction stays deterministic; creative writing varies; low resume temp minimises fabrication	Hand-tuned per node — another knob to maintain
LLM critic auto-retry capped at 3 (threshold 0.7), then always defer to human	Automated quality floor without unbounded spend or infinite loops	A fixed threshold is coarse; some borderline runs still reach the human
Deterministic-signals-first feedback classification	Approve flag can't be faked by text; per-document relevance check avoids regenerating on off-topic input	An extra LLM classification call per feedback box
In-memory session store + MemorySaver checkpointer	Zero infra; pause / resume across the HITL interrupt works out of the box	State lost on restart; single-worker only — no horizontal scale
Cursor-based polling instead of WebSockets	Simple client / server, proxy-friendly, easy to reason about	Slightly higher latency and repeated requests vs. push
DuckDuckGo search (no key) over a paid API	No key, no cost, runs anywhere	Thinner results than Tavily / SerpAPI; can be swapped in later

7. Technology Stack

Layer	Technology	Role
LLM	Configurable chat model via LangChain (LLM_MODEL)	Inference for all agent nodes (temp 0.0–0.6 per node)
Orchestration	LangGraph (StateGraph + MemorySaver)	Agent DAG + HITL checkpoint / resume
Web Framework	FastAPI + Uvicorn	REST API + cursor-based event streaming + background graph thread
Frontend	Single-page HTML (static/index.html)	Live node updates, per-document feedback cards, PDF export
Web Search	DuckDuckGo (via @tool)	Company research — no API key required
PDF I/O	pypdf (parse) · fpdf2 (export)	Extract text from uploaded resume / JD; render documents to PDF
Evaluation	deepeval (GEval / LLM-as-judge) + pytest	Quality, fabrication, and error-handling CI gates

8. API Reference

A FastAPI server drives the web UI: submit a run, poll for live node-completion events, and send approval or per-document feedback at the human-review interrupt. The agent graph runs on a background thread; the in-memory session store keys events and pending feedback by session_id.

Endpoint	Method	Description
/	GET	Serve the single-page web UI
/api/submit	POST	Start an agent run from a JD + resume; returns a session_id
/api/status/{session_id}	GET	Cursor-based event polling for real-time node updates
/api/feedback/{session_id}	POST	Submit global approval or per-document feedback at the interrupt
/api/parse-pdf	POST	Extract plain text from an uploaded resume or JD PDF
/api/export-pdf	POST	Render a markdown document to a downloadable PDF